Milestone 1 Report

Smart MCQ Solver — TF-IDF Baseline & MAP@3 Evaluation

A fully explained walkthrough of the milestone-1.ipynb notebook: every question, the reasoning behind each approach, the exact code used, the notebook's output, and the final answer — written so that anyone unfamiliar with the notebook can understand what was done and why.

Source notebook: milestone-1.ipynb
Dataset: Kaggle "smart-mcq-solver-challenge"

1. Project Overview

This notebook establishes the **baseline** for a multiple-choice question answering (MCQ) problem — each row in the dataset provides a question (**prompt**) and five candidate options (**A** through **E**), and the goal is to identify the correct option (**answer**). The notebook is organized as a guided sequence of 10 questions that build up purely classical, lexical NLP techniques — text cleaning, vocabulary analysis, stop-word filtering, and TF-IDF cosine-similarity — culminating in a complete ranking pipeline scored with the competition's MAP@3 metric. This establishes the performance floor that later, more sophisticated milestones (e.g. semantic sentence embeddings and transformer models) are measured against.

1.1 Environment & Setup

The notebook begins by verifying the software and hardware environment it is running in, then confirms that the three competition data files are present on disk before any analysis begins.

Python	3.10.12
Platform	Linux 6.6.114.1-microsoft-standard-WSL2 (WSL2)
NumPy	2.2.6
Pandas	2.3.3
PyTorch	2.5.1+cu121
Transformers	5.12.0
Device	CUDA (NVIDIA GeForce RTX 3050 Laptop GPU, ~4.29 GB VRAM)

1.2 Dataset

The dataset is read from three CSV files with pandas. A quick inspection with **.describe()**, **.shape** and **.isna().sum()** confirms the data is clean (no missing values) before any modeling begins.

Files	train.csv, test.csv, sample_submission.csv
train.csv shape	2000 rows × 8 columns
test.csv shape	500 rows × 7 columns
sample_submission.csv shape	500 rows × 2 columns
Columns	id, prompt, A, B, C, D, E, answer (test.csv omits answer)
Missing values	None — all columns fully populated across train.csv

2. Table of Contents

Text Preprocessing & Vocabulary Analysis

Q1. Frequency Distribution of the Correct Answer Option

Q2. Vocabulary Size After Lowercasing and Punctuation Removal

Q3. Stop-Word Filtering for a Single Prompt (Row ID 1)

TF-IDF Vectorization & Cosine Similarity

Q4. Fitting a TF-IDF Vectorizer Over Prompts and Options

Q5. Cosine Similarity Between a Prompt and Option A (Row ID 1)

Q6. Top-1 Accuracy of Pure TF-IDF Similarity Across the Whole Dataset

Understanding the MAP@3 Evaluation Metric

Q7. MAP@3 Worked Example — Correct Answer Ranked First

Q8. MAP@3 Worked Example — Correct Answer Ranked Second

Q9. The Majority-Class Baseline MAP@3 Score

End-to-End TF-IDF Baseline Pipeline

Q10. The Full TF-IDF Cosine-Similarity Ranking Pipeline (MAP@3)

3. Text Preprocessing & Vocabulary Analysis

Question 1: Frequency Distribution of the Correct Answer Option

Calculate the frequency distribution of the correct answer (A, B, C, D, E) in train.csv. Based on your counts, what is the sum of the occurrences of the most frequent option and the least frequent option?

Approach & Explanation

- Before building any model it is good practice to check whether the target label is balanced. **train["answer"].value_counts()** groups the 2000 rows by their **answer** letter (A–E) and counts how many times each one occurs, returning the counts sorted from most to least frequent.
- **.max()** and **.min()** on that count series pick out the single most common and single least common answer option's occurrence counts, regardless of which letter they belong to.
- Adding those two counts together gives a quick, single-number sense of how skewed the class distribution is: if the classes were perfectly balanced, the most- and least-frequent counts would be equal (2000 / 5 = 400 each, summing to 800); the actual sum of 814 shows the distribution is only mildly imbalanced.

Code

```
freq = train["answer"].value_counts()
max_fre = freq.max()
min_fre = freq.min()
#print(freq, max_fre, min_fre)
print("Sum of occurrences of the most frequent option and the least frequent option:",
    max_fre + min_fre)
```

Notebook Output

```
Sum of occurrences of the most frequent option and the least frequent option: 814
```

Sum of most-frequent + least-frequent answer counts = 814

Question 2: Vocabulary Size After Lowercasing and Punctuation Removal

After converting the prompt column to lowercase and removing all standard punctuation characters (using Python's string.punctuation), split the text by whitespace. What is the total number of unique words (vocabulary size) across the entire cleaned prompt column of train.csv?

Approach & Explanation

- **str.maketrans('', '', string.punctuation)** builds a translation table that maps every standard punctuation character (from Python's built-in **string.punctuation** constant, e.g. **! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [\ ] ^ _ ` { | } ~**) to nothing, effectively deleting it when applied with **.translate()**.
- For every prompt, the code lowercases the text with **.lower()**, strips punctuation with **.translate(translator)**, and then calls **.split()** to break the cleaned string into whitespace-separated word tokens.
- A Python **set()** called **vocab** accumulates every distinct word seen across all 2000 prompts via **.update()**, so any word repeated across multiple prompts is only counted once. Sets automatically deduplicate, which is exactly what is needed for a vocabulary count.
- **len(vocab)** at the end gives the total number of unique whitespace-separated, punctuation-free, lowercase words across the whole cleaned prompt column — this is the naive "bag of words" vocabulary size before any smarter tokenization (sub-word splitting, stop-word removal, etc.) is applied.

Code

```
import string

#help(str.maketrans)
#print("String punctuation characters:", string.punctuation)

# Remove punctuation and convert to lowercase
translator = str.maketrans('', '', string.punctuation)

#table = str.maketrans("abc", "xyz")
#text = "apple banana cat"
#print(text.translate(table))

vocab = set() # for unique characters

for text in train["prompt"].astype(str):
    cleaned_text = text.lower().translate(translator)
    vocab.update(cleaned_text.split())

print("Length of unique characters in the vocabulary:", len(vocab))
#print("Unique characters in the vocabulary:", sorted(vocab))
```

Notebook Output

```
Length of unique characters in the vocabulary: 859
```

Unique cleaned-word vocabulary size across all prompts = 859

Question 3: Stop-Word Filtering for a Single Prompt (Row ID 1)

Using the cleaned prompt from Row ID 1, filter out the standard English stop words using sklearn.feature_extraction.text.ENGLISH_STOP_WORDS. How many words are left in the prompt for Row ID 1 after filtering?

Approach & Explanation

- Stop words ("the", "is", "a", "of", "what", …) are extremely common words that carry little topic-specific meaning; removing them is a standard step before computing word-overlap or TF-IDF style similarity so that matches are driven by meaningful content words instead.
- **ENGLISH_STOP_WORDS** is a fixed, built-in frozenset of ~318 common English stop words shipped with scikit-learn — no fitting or configuration is required, it is simply imported and used directly.
- The row for **id == 1** is located with boolean indexing (**train.loc[train["id"]==1, "prompt"]**), and the same lowercase + punctuation-stripping cleanup from Q2 is applied to its prompt text before splitting it into words.
- A list comprehension keeps only the words that are *not* present in **ENGLISH_STOP_WORDS**, and **len()** of the resulting list counts how many content words remain. For this prompt ("Pick the best possible answer: What is Martin Heidegger's view on the relationship between time and human existence? among the listed options."), filler words like "the", "is", "on", "between" and "among" are dropped, leaving 13 substantive words.

Code

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

prompt = train.loc[train["id"]==1, "prompt"].iloc[0]

translator = str.maketrans("", "", string.punctuation)

cleaned_prompt = prompt.lower().translate(translator)
words = cleaned_prompt.split()
filter_words = [word for word in words if word not in ENGLISH_STOP_WORDS]

print(f"Original prompt:\n{prompt}\n")
print(f"Filtered words:\n{filter_words}\n")

print("Count of filtered words:", len(filter_words))
```

Notebook Output

```
Original prompt:
Pick the best possible answer: What is Martin Heidegger's view on the
relationship between time and human existence? among the listed options.

Filtered words:
['pick', 'best', 'possible', 'answer', 'martin', 'heideggers', 'view',
'relationship', 'time', 'human', 'existence', 'listed', 'options']

Count of filtered words: 13
```

Words remaining in Row ID 1's prompt after stop-word removal = 13

4. TF-IDF Vectorization & Cosine Similarity

Question 4: Fitting a TF-IDF Vectorizer Over Prompts and Options

Fit a default TfidfVectorizer(stop_words='english') on a list containing all the combined text of the prompts and options in train.csv. What is the exact total number of feature columns (vocabulary size) generated by the vectorizer?

Approach & Explanation

- TF-IDF (Term Frequency – Inverse Document Frequency) turns text into numeric vectors by weighting each word by how often it appears in a given document relative to how common it is across all documents — words that appear everywhere (and are therefore less informative) get down-weighted automatically, which is why this is a step up from raw word counting.
- **combined_text** concatenates the **prompt** and all five options (A through E) for every row into one long string per row, so the vectorizer's vocabulary is learned from the full pool of question-and-answer text rather than just the questions.
- **TfidfVectorizer(stop_words="english")** automatically lowercases text, tokenizes it, strips scikit-learn's built-in English stop-word list, and learns a fixed vocabulary of all remaining unique terms when **.fit_transform()** is called.
- The resulting sparse matrix **x** has shape (num_rows, vocabulary_size); here that is **(2000, 2762)**, meaning the vectorizer discovered 2,762 distinct vocabulary terms across all 2000 rows' combined prompt+option text — confirmed directly by **len(vectorizer.get_feature_names_out())**.

Code

```
from sklearn.feature_extraction.text import TfidfVectorizer

combined_text = (
    train["prompt"].fillna("")  + " "+
    train["A"].fillna("") + " " +
    train["B"].fillna("") + " " +
    train["C"].fillna("") + " " +
    train["D"].fillna("") + " " +
    train["E"].fillna("")
    )

vectorizer = TfidfVectorizer(stop_words="english")
x = vectorizer.fit_transform(combined_text)

print("Shape:", x.shape)
print("Vocabulary size:", len(vectorizer.get_feature_names_out()))
```

Notebook Output

```
Shape: (2000, 2762)
Vocabulary size: 2762
```

TF-IDF vectorizer vocabulary size (fit on prompts + all options) = 2,762 features

Question 5: Cosine Similarity Between a Prompt and Option A (Row ID 1)

Using the TF-IDF vectorizer fitted in Question 4, calculate the cosine similarity between the prompt and option A strictly for Row ID 1. What is the resulting similarity score? (Round to 4 decimal places).

Approach & Explanation

- This question reuses the exact same **vectorizer** object fitted in Q4 — it is not re-fit here — because the whole point of TF-IDF similarity comparisons is that every piece of text is projected into the *same* fixed vocabulary space so the resulting vectors are directly comparable.
- The row with **id == 1** is located, and its **prompt** and **A** text are each transformed into TF-IDF vectors with **vectorizer.transform()** (note: **.transform()**, not **.fit_transform()**, since the vocabulary is already learned).
- **cosine_similarity()** from scikit-learn computes the cosine of the angle between the two sparse vectors, returning a value from 0 (no shared weighted terms) to 1 (identical direction in TF-IDF space). The function returns a 2-D array even for single-vector inputs, so **[0][0]** extracts the single scalar score.
- The result, 0.2720, indicates only modest lexical overlap between the question and option A for this particular row — a first hint, revisited in Q6, that raw word-overlap similarity is a fairly weak signal for picking the correct MCQ answer on its own.

Code

```
from sklearn.metrics.pairwise import cosine_similarity

row = train.loc[train["id"] == 1].iloc[0]

prompt = row["prompt"]
option_a = row["A"]


prompt_vec = vectorizer.transform([prompt])
option_a_vec = vectorizer.transform([option_a])

similarity = cosine_similarity(prompt_vec, option_a_vec)[0][0]

print(similarity)
print(round(similarity, 4))
```

Notebook Output

```
0.27202429519891635
0.272
```

Cosine similarity (prompt, Option A) for Row ID 1 = 0.2720

Question 6: Top-1 Accuracy of Pure TF-IDF Similarity Across the Whole Dataset

Expand the logic from Question 5: For every row in train.csv, calculate the cosine similarity between the prompt and each of its 5 options. Then calculate the percentage of instances where the option with the highest cosine similarity matches the correct answer.

Approach & Explanation

- This scales Q5's single-row similarity calculation up to the full training set: for every one of the 2000 rows, the prompt is compared against all five options (A–E), giving five similarity scores per row.
- **scores.index(max(scores))** finds the position of the highest score among the five, and indexing into **options** at that position converts it back into the corresponding letter — this is the model's single "best guess" for that row, analogous to Top-1 (not Top-3) prediction.
- Comparing that predicted letter against the row's true **answer** column and counting matches gives a straightforward accuracy calculation: **correct / len(train) * 100**.
- The resulting 13.55% accuracy is barely better than the 20% one would expect from random guessing among 5 options — actually noticeably worse. This is an important and slightly counter-intuitive baseline result: purely lexical, word-overlap-based similarity is a poor standalone predictor of the correct MCQ answer, since the correct answer often does not share very much surface-level vocabulary with the question. It sets up the motivation for the smarter approaches (MAP@3-based Top-3 evaluation and, in later milestones, semantic embeddings) explored in the rest of the notebook.

Code

```
options = ["A", "B", "C", "D", "E"]

correct = 0

for _, row in train.iterrows():

    prompt_vec = vectorizer.transform([str(row["prompt"])])

    scores = []

    for opt in options:
        score = cosine_similarity(prompt_vec, vectorizer.transform([str(row[opt])]))[0][0]
        scores.append(score)

    predicted = options[scores.index(max(scores))]

    if predicted == row["answer"]:
        correct += 1

accuracy = correct / len(train)*100
print(f"Accuracy: {accuracy:.2f}%")
```

Notebook Output

```
Accuracy: 13.55%
```

Top-1 accuracy of pure TF-IDF cosine-similarity matching = 13.55%

5. Understanding the MAP@3 Evaluation Metric

Question 7: MAP@3 Worked Example — Correct Answer Ranked First

If the ground truth answer for a question is C, what is the MAP@3 score if a model predicts C A B? (Mean Average Precision at 3.)

Approach & Explanation

- MAP@3 is the competition's core evaluation metric for this MCQ task. For a single question, the *Average Precision at 3* (AP@3) score is 1 divided by the rank (1-indexed) at which the correct answer first appears among the top 3 predictions, or 0 if it does not appear in the top 3 at all. The **ap_at_3()** helper function defined here implements exactly this rule and is reused throughout the rest of the notebook (Q8, Q9, Q10 and beyond).
- Here the model's ranked guesses are **[C, A, B]** and the true answer is **C**. Since **C** is the very first (rank 1) prediction, **predicted[:3].index("C")** returns 0, and the score is **1 / (0 + 1) = 1.0** — the maximum possible AP@3 score, reflecting that getting the correct answer right on the first guess is rewarded fully.

Code

```
def ap_at_3(actual, predicted):
    if actual in predicted[:3]:
        return 1 / (predicted[:3].index(actual) + 1)
    return 0

actual = "C"
predicted = ["C", "A", "B"]

print(f"AP@3: {ap_at_3(actual, predicted):.2f}")
```

Notebook Output

```
AP@3: 1.00
```

AP@3 for actual=C, predicted=[C, A, B] = 1.00 (correct answer ranked 1st)

Question 8: MAP@3 Worked Example — Correct Answer Ranked Second

If the ground truth answer for a question is B, what is the MAP@3 score if a model predicts D B E?

Approach & Explanation

- This reuses the same **ap_at_3()** function from Q7 with a different scenario to illustrate how the score degrades gracefully rather than dropping straight to zero when the correct answer isn't the very first guess.

- The model's ranked guesses are **[D, B, E]** and the true answer is **B**. **B** is found at index 1 (i.e. rank 2, the second guess), so the score is **1 / (1 + 1) = 0.5**.
- This demonstrates the core intuition behind MAP@3: being right still counts even if it isn't your first guess, but you're rewarded less the further down your ranked list the correct answer falls — a much more forgiving and informative metric than plain Top-1 accuracy (as used in Q6) when a model is allowed to submit multiple ranked guesses.

Code

```
actual = "B"
predicted = ["D", "B", "E"]

print(f"AP@3: {ap_at_3(actual, predicted):.2f}")
```

Notebook Output

```
AP@3: 0.50
```

AP@3 for actual=B, predicted=[D, B, E] = 0.50 (correct answer ranked 2nd)

Question 9: The Majority-Class Baseline MAP@3 Score

The Majority Class Baseline: Find the most frequent correct answer in the training set (using your data from Q1). Make a static prediction for every single row where that most frequent answer is your 1st guess, followed by the second most frequent, and then the third most frequent. What is the overall MAP@3 score of this "Majority Class" baseline on train.csv?

Approach & Explanation

- A majority-class (or "dumbest possible") baseline ignores the actual question text entirely and always predicts the same fixed ranking of options for every row, based purely on how often each option was correct in the training data overall. Every serious model should be expected to clearly beat this baseline.
- **freq** (computed back in Q1 via **train["answer"].value_counts()**) is already sorted from most to least frequent, so **freq.index[:3]** directly gives the three most common answer letters in rank order — this fixed 3-item list, **top3**, is used as the identical Top-3 prediction for every single row in the dataset.
- For each row, **ap_at_3(actual, top3)** (the same helper from Q7/Q8) scores how well that same static guess performs against that row's true answer, and the scores are averaged across all 2000 rows to get the overall MAP@3.
- The result, roughly 0.4213, is a meaningful reference point: it shows that just knowing the overall class frequencies (with zero understanding of the actual question) already yields a moderate MAP@3, both because the classes are only mildly imbalanced (from Q1) and because MAP@3 credits partial correctness at ranks 2 and 3. Any genuinely useful model (like the TF-IDF pipeline in Q10) needs to beat this baseline by a meaningful margin to prove it is learning something from the question content.

Code

```
top3 = freq.index[:3].tolist()

scores = []

for actual in train["answer"]:
    scores.append(
        ap_at_3(actual, top3)
    )

overall_map3 = sum(scores) / len(scores)

print("MAP@3 =", overall_map3)
```

Notebook Output

```
MAP@3 = 0.4212500000000017
```

Majority-class baseline MAP@3 (same static Top-3 guess for every row) = 0.4213

6. End-to-End TF-IDF Baseline Pipeline

Question 10: The Full TF-IDF Cosine-Similarity Ranking Pipeline (MAP@3)

The TF-IDF Pipeline: Build a basic pipeline that evaluates every row in train.csv. For each row, calculate the TF-IDF cosine similarity between the prompt and each of the 5 options. Sort these options from highest similarity to lowest to form your top 3 predictions. What is the final average MAP@3 score of this TF-IDF pipeline across the entire training set?

Approach & Explanation

- This question assembles everything from Q4–Q9 into one complete, self-contained baseline pipeline — it even re-loads **train.csv** and re-fits a fresh **TfidfVectorizer** from scratch so the whole pipeline can run independently of earlier cells.
- For every row, all five options are ranked by their TF-IDF cosine similarity to the prompt (the same per-option similarity computation as Q5/Q6), and **similarities.sort(key=lambda x: x[1], reverse=True)** orders them from most to least similar.
- Unlike Q6 (which only kept the single top guess for a plain accuracy score), this pipeline keeps the **top3** highest-similarity options as a ranked list of three guesses per row, matching the actual MAP@3 competition format.
- Each row's AP@3 is computed inline (equivalent to calling **ap_at_3()**: full credit 1.0 if the correct answer is the top guess, 0.5 if second, 0.333 if third, 0 if it falls outside the top 3), and the mean across all 2000 rows gives the final pipeline MAP@3 of **0.2962**.
- Interestingly, this full ranking-based MAP@3 (0.2962) is noticeably *lower* than the naive majority-class baseline from Q9 (0.4213) — a useful, slightly humbling reminder that a content-aware method is not automatically better than a content-blind one, and that TF-IDF's purely lexical similarity is a genuinely weak signal for this task. This exact gap is what motivates moving to semantic sentence embeddings (MiniLM) in Milestone 2, which was shown there to lift MAP@3 to 0.4231.

Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

train = pd.read_csv("../dataset/smart-mcq-solver-challenge/train.csv")

# Fit TF-IDF on all prompt+option text
combined_text = (
    train["prompt"].fillna("") + " " +
    train["A"].fillna("") + " " +
    train["B"].fillna("") + " " +
    train["C"].fillna("") + " " +
    train["D"].fillna("") + " " +
    train["E"].fillna("")
)

vectorizer = TfidfVectorizer(stop_words="english")
```

```
vectorizer.fit(combined_text)

options = ["A", "B", "C", "D", "E"]

scores = []

for _, row in train.iterrows():

    prompt_vec = vectorizer.transform([str(row["prompt"])])

    similarities = []

    for opt in options:

        option_vec = vectorizer.transform([str(row[opt])])

        sim = cosine_similarity(
            prompt_vec,
            option_vec
        )[0][0]

        similarities.append((opt, sim))

    # Sort by similarity descending
    similarities.sort(key=lambda x: x[1], reverse=True)

    top3 = [x[0] for x in similarities[:3]]

    actual = row["answer"]

    if actual in top3:
        ap3 = 1.0 / (top3.index(actual) + 1)
    else:
        ap3 = 0.0

    scores.append(ap3)

map3 = sum(scores) / len(scores)

print("MAP@3 =", map3)
print("Rounded =", round(map3, 4))
```

Notebook Output

```
MAP@3 = 0.29616666666666625
Rounded = 0.2962
```

Full TF-IDF cosine-similarity ranking pipeline MAP@3 = 0.2962

7. Summary of Answers

The table below collects every question's final answer in one place for quick reference.

Q#	Answer
Q1	Sum of most-frequent + least-frequent answer counts = 814
Q2	Unique cleaned-word vocabulary size across all prompts = 859
Q3	Words remaining in Row ID 1's prompt after stop-word removal = 13
Q4	TF-IDF vectorizer vocabulary size (fit on prompts + all options) = 2,762 features
Q5	Cosine similarity (prompt, Option A) for Row ID 1 = 0.2720
Q6	Top-1 accuracy of pure TF-IDF cosine-similarity matching = 13.55%
Q7	AP@3 for actual=C, predicted=[C, A, B] = 1.00 (correct answer ranked 1st)
Q8	AP@3 for actual=B, predicted=[D, B, E] = 0.50 (correct answer ranked 2nd)
Q9	Majority-class baseline MAP@3 (same static Top-3 guess for every row) = 0.4213
Q10	Full TF-IDF cosine-similarity ranking pipeline MAP@3 = 0.2962

Key Takeaways

- The correct-answer distribution across A–E is only mildly imbalanced (most- + least-frequent counts sum to 814 out of 2000×2 = 4000 possible option-slots), so a naive majority-class guess is a non-trivial baseline to beat.
- Basic text cleaning (lowercasing + punctuation stripping) reveals a raw vocabulary of 859 unique words across all prompts; stop-word filtering on a single prompt further trims it down to only the content-bearing words (13 of the original words for Row ID 1).
- A default TF-IDF vectorizer fit on prompts + all five options learns a 2,762-term vocabulary; the resulting vectors let cosine similarity be used as a simple, fine-tuning-free lexical relevance score between a question and each candidate answer.
- Picking the single option with the highest TF-IDF similarity as a Top-1 guess is surprisingly weak (13.55% accuracy — worse than the 20% expected from random guessing among 5 options), showing that raw word overlap alone is a poor signal for this task.
- MAP@3 rewards a correct answer more the higher it is ranked (1.0 / 0.5 / 0.33 for ranks 1 / 2 / 3, 0 otherwise) rather than requiring a single perfect guess, which is why even the content-blind majority-class baseline scores a respectable 0.4213.

- The full TF-IDF Top-3 ranking pipeline actually scores lower (MAP@3 = 0.2962) than the naive majority-class baseline (0.4213) — a clear signal that this classical lexical approach is not enough on its own, motivating the move to semantic sentence embeddings (MiniLM) explored in Milestone 2, which lifted MAP@3 to 0.4231.